Square Root Decomposition

Complete algorithmic notes • simple English • JavaScript examples

Core idea: Divide an array into blocks of about √n elements. Store a summary for each block, then reuse those summaries to avoid scanning every element for every query.

1. What problem does it solve?

Suppose an array supports two operations: **update(index, val)**, which changes one element, and **sumRange(left, right)**, which returns the sum between two indices. A normal range-sum loop can take O(n) time per query. If there are many queries, repeatedly adding the same elements wastes time.

Square Root Decomposition stores the sum of each block. A query can add a few boundary elements directly and use precomputed sums for complete blocks in the middle.

2. Split the array into blocks

Let n be the number of elements. Choose a block size B ≈ √n, commonly B = ceil(√n). The number of blocks is ceil(n / B).

```
this.blockSize = Math.ceil(Math.sqrt(nums.length));
this.blocks = new Array(
    Math.ceil(nums.length / this.blockSize)
).fill(0);
```

Example: n = 9, so B = 3. The array [2, 4, 6, 8, 10, 12, 14, 16, 18] is split into three blocks:

Block	Array indices	Values	Stored sum
0	0–2	2, 4, 6	12
1	3–5	8, 10, 12	30
2	6–8	14, 16, 18	48

The block-sum array is `blocks = [12, 30, 48]`. The original array stores individual values; the blocks array stores one summary per block.

Find the block for an index

```
let blockIndex = Math.floor(index / this.blockSize);
```

With block size 3, indices 0, 1, 2 belong to block 0; indices 3, 4, 5 belong to block 1; and indices 6, 7, 8 belong to block 2.

3. Build the block sums

```
for (let i = 0; i < nums.length; i = i + 1) {
    let blockIndex = Math.floor(i / blockSize);
    blocks[blockIndex] += nums[i];
}
```

Visit each element once and add it to its block's sum. Building all block sums takes O(n) time.

4. Update one element

If an element changes, we do not need to recalculate the entire block. Remove the old value from the block sum, add the new value, then update the array.

```
update(index, val) {
```

```
    let blockIndex = Math.floor(index / this.blockSize);

    this.blocks[blockIndex] -= this.nums[index];
    this.blocks[blockIndex] += val;
    this.nums[index] = val;
}
```

Example: if a block sum is 30 and its element 10 changes to 20, the new block sum is 30 - 10 + 20 = 40. This update takes O(1) time.

5. Range sum: the two cases

This is the key part. Find the block containing left and the block containing right. Then handle one of two cases.

Case 1 — both endpoints are in the same block

Sum the requested elements directly. The stored block sum includes elements outside the query, so it cannot be used as-is.

```
if (leftBlock === rightBlock) {
    for (let i = left; i <= right; i = i + 1) {
        sum += this.nums[i];
    }
}
```

Case 2 — endpoints are in different blocks

Split the requested range into three sections:

Section	What to do
Left partial block	Sum nums[left] through the end of the left block.
Complete middle blocks	Add each precomputed value from blocks[].
Right partial block	Sum from the start of the right block through nums[right].

Example: sumRange(1, 7) on [2, 4, 6, 8, 10, 12, 14, 16, 18], with block size 3:

```
Left partial block:   nums[1] + nums[2] = 4 + 6 = 10
Middle complete block: blocks[1] = 30
Right partial block:  nums[6] + nums[7] = 14 + 16 = 30

Total = 10 + 30 + 30 = 70
```

The middle-block loop uses block indices, not array indices:

```
for (let i = leftBlock + 1; i < rightBlock; i = i + 1) {
    sum += this.blocks[i];
}
```

6. Complete JavaScript implementation

```
class NumArray {
    constructor(nums) {
        this.nums = nums;
        this.n = nums.length;
        this.blockSize = Math.ceil(Math.sqrt(this.n));
        this.blocks = new Array(
            Math.ceil(this.n / this.blockSize)
        ).fill(0);

        // Build block sums
        for (let i = 0; i < this.n; i = i + 1) {
            let blockIndex = Math.floor(i / this.blockSize);
            this.blocks[blockIndex] += nums[i];
        }
    }

    update(index, val) {
        let blockIndex = Math.floor(index / this.blockSize);

        this.blocks[blockIndex] -= this.nums[index];
        this.blocks[blockIndex] += val;
        this.nums[index] = val;
    }

    sumRange(left, right) {
        let sum = 0;
        let leftBlock = Math.floor(left / this.blockSize);
        let rightBlock = Math.floor(right / this.blockSize);

        // Case 1: both endpoints are in the same block
        if (leftBlock === rightBlock) {
            for (let i = left; i <= right; i = i + 1) {
                sum += this.nums[i];
            }
        } else {
            // Sum the left partial block
            let leftBlockEndIndex =
                (leftBlock + 1) * this.blockSize - 1;

            for (let i = left; i <= leftBlockEndIndex; i = i + 1) {
                sum += this.nums[i];
            }

            // Add complete middle blocks
            for (let i = leftBlock + 1; i < rightBlock; i = i + 1) {
                sum += this.blocks[i];
            }

            // Sum the right partial block
            let rightBlockStartIndex = rightBlock * this.blockSize;

            for (let i = rightBlockStartIndex; i <= right; i = i + 1) {
                sum += this.nums[i];
            }
        }

        return sum;
    }
}
```

7. Complexity

Operation	Time
Build block sums	O(n)
Update one element	O(1)
sumRange, same block	O(B)
sumRange, different blocks	O(B + n/B)
sumRange when B ≈ √n	O(√n)
Extra space for block summaries	O(n/B), or O(√n)

Why √n? A range query does roughly B work for the two boundary blocks and n/B work for complete middle blocks. Choosing B near √n balances these costs.

8. When to use it

Useful when: there are frequent range queries and point updates; each block can store a useful summary; and O(√n) query time is acceptable.

Examples include range sums, some range minimum/maximum queries, frequency queries, and selected offline query problems. The exact block summary and update logic depend on the operation.

9. When not to use it

Consider alternatives when:

• There are very few queries; a direct loop may be simpler.

• The array never changes and you only need range sums; prefix sums usually give O(1) queries after O(n) preprocessing.

• You need faster query/update operations at scale; a Fenwick tree or segment tree may fit better.

• The query or update cannot be handled efficiently using a block summary.

10. Mental model to remember

• **Divide:** split the array into blocks of about √n elements.

• **Precompute:** store a summary, such as each block's sum.

• **Update:** adjust the summary for the block containing the changed element.

• **Query:** handle boundary elements directly and use summaries for complete blocks in the middle.

Key idea: Square Root Decomposition is not just about splitting an array into chunks. It is about using precomputed block information to skip repeated work while handling the query boundaries correctly.